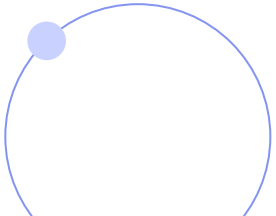
Carga de datos

Funciones de carga de datos

Hasta el momento, hemos usado la instrucción `print` para imprimir información en la consola. La operación inversa es la **entrada de datos**, es decir que **el usuario ingrese información en la consola y podamos guardarla en una variable**. En Python, esto se realiza vía la instrucción **`input`**.

```
nombre = input("Escribe tu nombre: ")
```

Notarás que `input` es similar a `print`, puesto que **lo que indicamos entre paréntesis y entre comillas será mostrado en pantalla**. En el caso de `input`, decimos que "**retorna**" un valor (así como 7 + 5 retornaba 12), que no es otra cosa que lo escrito en la consola por el usuario, y lo almacenamos en una variable que queramos.



Una vez ingresado el nombre, podemos imprimirlo en pantalla:

```
nombre = input("Escribe tu nombre: ")
print(nombre)
```



También, como el **resultado de `input` es siempre una cadena (cualquiera sea el tipo de valor que se ingrese),** podemos usar el operador de suma para mostrar un saludo:

```
nombre = input("Escribe tu nombre:")
print("Hola " + nombre)
```

educación IT
CODE YOUR FUTURE

Introducción a las funciones de conversión

¿En qué consiste la conversión?

Es una acción común y muy sencilla que consiste en **convertir un tipo de dato a otro**. Como vemos en el ejemplo, podemos convertir un número de coma flotante a un número entero mediante la instrucción **`int`**.

Como los números enteros no pueden tener parte decimal, convertir un número de coma flotante a un número entero implica despreciar los decimales.

```
>>> pi = 3.14
>>> pi_entero = int(pi)
>>> pi_entero
3
```



Hay una conversión en particular que es la que más nos interesa y la más empleada: la **conversión de una cadena a un número entero** y viceversa.

Para el primer caso, usamos la instrucción **`int`**:

```
>>> numero_en_cadena = "30"
>>> numero_entero = int(numero_en_cadena)
>>> numero_entero
30
>>> type(numero_en_cadena)
<class 'str'>
>>> type(numero_entero)
<class 'int'>
```

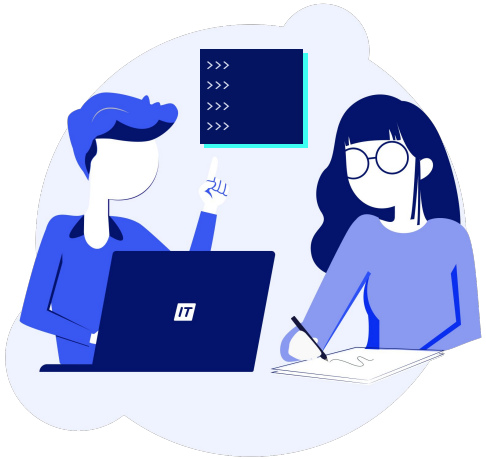
Para la **conversión de un número entero a una cadena**, usamos **`str`**:

```
>>> numero_entero = 30
>>> numero_en_cadena = str(numero_entero)
>>> numero_en_cadena
'30'
>>> type(numero_en_cadena)
<class 'str'>
```

La diferencia entre tener un número almacenado como una cadena o como un entero **radica en las operaciones que queremos ejecutar sobre ellos**.

Si queremos sumar el número 30 a otro número, será mejor tenerlo almacenado como un número entero (puesto que las operaciones aritméticas solo funcionan sobre números enteros y de coma flotante).

Por otro lado, si necesitamos el número 30 para imprimir en pantalla “Tu edad es: 30”, entonces conviene tenerlo como una cadena, para poder realizar la concatenación con el operador **+**.



Observa este ejemplo:

```
>>> numero_treinta = 30
>>> cadena_treinta = "30"
>>> # Permitido: suma entre dos enteros.
... numero_treinta + 10
40
>>> # No permitido: suma entre una cadena y un entero.
... cadena_treinta + 10
[Error]
>>> # Permitido: suma entre dos cadenas (concatenación).
... print("Tu edad es: " + cadena_treinta)
Tu edad es: 30
... # No permitido: suma entre una cadena y un entero.
>>> print("Tu edad es: " + numero_treinta)
[Error]
```

¿Por qué son importantes las funciones de conversión?

Las funciones de conversión son esenciales para **transformar datos entre diferentes tipos**, como convertir una cadena en un número entero o un número decimal en una cadena. Estas conversiones **permiten manejar datos de manera más flexible y adecuada según el contexto**.

Además, garantizan que los programas sean más robustos al **evitar errores que surgen al intentar operar con tipos incompatibles**.



Analicemos este **ejemplo**: Un programa que solicita al usuario dos números para sumarlos. Dado que la entrada del usuario siempre se recibe como texto, es necesario convertirla a un tipo numérico para realizar la operación.

```
# Solicitar al usuario dos números
num1 = input("Ingresa el primer número: ")
num2 = input("Ingresa el segundo número: ")

# Convertir las entradas a enteros para realizar la suma
num1 = int(num1)
num2 = int(num2)

# Calcular la suma
resultado = num1 + num2

# Mostrar el resultado
print("La suma de los números es:", resultado)
```

En este caso, la **función `int()` convierte las entradas del usuario (que son cadenas de texto) en números enteros**. Sin esta conversión, intentar sumar las entradas como texto concatenaría los valores en lugar de sumarlos matemáticamente, lo que generaría un resultado incorrecto. Este proceso asegura que los datos sean del tipo adecuado para la operación deseada.

